

南京邮电大学

毕业设计（论文）

题 目 数组类型变量状态引导的模糊测试工具的设计与实现

专 业 软件工程

学生姓名 孙琳

班级学号 B220422 B22042210

指导老师 王子元

指导单位 计算机学院、软件学院、网络空间安全学院

日期： 2025 年 12 月 15 日至 2026 年 5 月 20 日

毕业设计（论文）原创性声明

本人郑重声明：所提交的毕业设计（论文），是本人在导师指导下，独立进行研究工作所取得的成果。除文中已注明引用的内容外，本毕业设计（论文）不包含任何其他个人或集体已经发表或撰写过的作品成果。对本研究做出过重要贡献的个人和集体，均已在文中以明确方式标明并表示了谢意。

论文作者签名：

日期： 年 月 日

摘 要

在众多软件测试技术中，模糊测试凭借着自动化程度高、漏洞发现能力强、实现成本低等优势，成为广泛关注的研究热点。同时，随着近些年来 AFL、LibFuzzer 等开源工具的普及，模糊测试技术得到快速发展与广泛应用。然而，传统模糊测试多采用以覆盖率增长为核心的导向策略，通过代码覆盖率、边覆盖率等指标来评估测试用例质量，并优先选择能够提升覆盖率的种子进行变异。该策略的确能够有效拓展代码覆盖广度，但难以有效触发与特定变量状态相关的程序错误。

数组作为基础且重要的数据结构，在开发过程中被大量使用，因此数组相关的安全问题也长期是软件漏洞的重要来源之一。据 CWE 数据库统计，CWE-119（缓冲区错误）、CWE-125（越界读取）、CWE-787（越界写入）等数组操作类漏洞，长期位居高频软件漏洞前列。现有模糊测试工具在处理数组相关代码时，通常将数组视为普通内存区域，缺乏对数组变量状态的精细化建模与针对性引导，难以有效发现与数组状态相关的漏洞。

针对上述问题，本文提出一种基于数组类型变量状态引导的模糊测试方法。该方法在现有模糊测试框架基础上，引入数组变量状态多样化与目标状态作为引导机制，通过对数组变量状态进行精细化建模与分析，设计状态多样性度量方法与面向数组状态的测试用例生成策略，从而提升模糊测试对数组相关漏洞的触发与检测能力。本文基于 AFL++ 框架实现了原型工具，设计了 LLVM Pass 进行数组访问的静态分析与动态插桩，实现了运行时数组状态采集与追踪模块，并设计了面向数组边界的自定义变异器。实验结果表明，该工具能够有效识别数组访问操作，准确采集数组运行时状态信息，并在短时间内成功触发数组越界类漏洞。

关键词： 模糊测试；数组状态；状态引导；灰盒测试；AFL

ABSTRACT

Among various software testing techniques, fuzzing has gradually become a research hotspot widely concerned by industry and academia due to its high automation, strong vulnerability discovery capability and low implementation cost. In recent years, with the popularization of open-source fuzzing tools such as AFL and LibFuzzer, fuzzing technology has achieved rapid development and wide application. However, traditional fuzzing mainly adopts a coverage-guided strategy, which evaluates the quality of test cases through indicators such as code coverage and edge coverage, and preferentially selects seeds that can increase coverage for mutation. Although this coverage-oriented strategy can improve the breadth of code coverage, it performs poorly in triggering errors related to specific variable states.

As a basic and important data structure, arrays are widely used in various software programs. Nevertheless, array-related security issues have long been a major source of software vulnerabilities. According to the CWE database, array operation-related vulnerabilities such as CWE-119 (Buffer Error), CWE-125 (Out-of-bounds Read) and CWE-787 (Out-of-bounds Write) have long been at the forefront of the most common software vulnerabilities. Existing fuzzing tools usually treat arrays as ordinary memory regions when processing array-related code, lacking refined modeling and guidance of array variable states, resulting in failure to effectively detect many array state-related vulnerabilities.

To address the above problems, this paper proposes a fuzzing method guided by array-type variable states. Based on existing fuzzing frameworks, this method introduces the diversification and specific states of array variables as the guidance mechanism. Through refined modeling and analysis of array variable states, it designs a state diversity metric method and a state-based test case generation strategy, thereby improving the ability of fuzzing to trigger array-related errors. This paper implements a prototype tool based on the AFL++ framework, designs an LLVM Pass for static analysis and dynamic instrumentation of array accesses, implements a runtime array state collection and tracking module, and designs a custom mutator oriented toward array boundaries. Experimental results show that the tool can effectively identify array access operations, accurately collect array runtime state information, and successfully trigger array out-of-bounds vulnerabilities in a short period of time.

Keywords: Fuzzing; Array State; State-guided; Grey-box Testing;

AFL

目 录

第一章 绪论	1
1.1 选题背景与意义	1
1.2 国内外研究现状	1
1.2.1 传统覆盖率引导模糊测试研究	1
1.2.2 状态感知与定向模糊测试研究	2
1.2.3 现有研究不足	2
1.3 主要工作	2
1.4 论文结构安排	3
第二章 相关理论与技术基础	4
2.1 模糊测试基本原理	4
2.1.1 模糊测试定义与流程	4
2.1.2 AFL 工作原理与架构	4
2.2 数组类型安全漏洞分析	5
2.2.1 典型数组漏洞类型	5
2.2.2 数组漏洞触发条件与特征	5
2.3 程序状态建模与动态插桩技术	6
2.3.1 程序运行时状态分析	6
2.3.2 LLVM 动态插桩技术	6
2.4 本章小结	7
第三章 数组类型变量状态引导模糊测试方法设计	8
3.1 总体设计方案	8
3.2 数组变量运行时状态识别方法	8
3.2.1 静态分析：数组属性提取	9
3.2.2 动态插桩：数组运行时状态采集	9
3.2.3 状态信息存储结构	10
3.3 数组状态多样性度量模型	10
3.3.1 状态向量定义	10
3.3.2 状态多样性度量算法	10
3.3.3 未探索状态区域识别	11
3.4 基于数组状态的测试用例生成策略	11
3.4.1 数组专用变异算子	11
3.4.2 两阶段定向变异流程	11
3.4.3 自适应变异参数调整	12
3.5 状态与覆盖率融合评估机制	12
3.6 本章小结	12

第四章 模糊测试工具的实现	13
4.1 开发环境与总体架构	13
4.2 数组状态采集模块实现	13
4.2.1 静态分析模块实现	13
4.2.2 动态插桩模块实现	14
4.3 状态度量与反馈模块实现	14
4.3.1 状态计算模块实现	14
4.3.2 反馈控制逻辑实现	14
4.4 测试用例生成与变异模块实现	15
4.4.1 基础变异逻辑实现	15
4.4.2 数组定向变异实现	15
4.5 工具集成与工作流程	15
4.5.1 模块集成方案	15
4.5.2 工具使用流程	15
4.6 本章小结	16
第五章 实验设计与结果分析	17
5.1 实验环境与测试目标	17
5.2 实验方案设计	17
5.2.1 实验一：状态识别有效性验证	17
5.2.2 实验二：状态多样性度量效果	17
5.2.3 实验三：数组漏洞发现能力验证	18
5.2.4 实验四：性能开销测试	18
5.3 实验结果与分析	18
5.3.1 状态采集准确率	18
5.3.2 状态空间探索效率	18
5.3.3 数组漏洞发现能力	19
5.3.4 执行效率与性能开销	19
5.4 本章小结	20
结束语	21
结束语	22
致谢	23
致谢	24
参考文献	25
附录	26

第一章 绪论

1.1 选题背景与意义

软件安全是信息系统稳定运行的重要保障，漏洞挖掘与检测技术则是维护软件安全的关键环节。在各类软件缺陷中，数组越界、缓冲区溢出、非法内存访问等问题极易引发程序崩溃、数据泄露乃至系统被入侵，是长期威胁软件安全的重要隐患^[1]。因此，针对此类漏洞开展高效、精准的检测技术研究，具有重要的现实价值。

在众多漏洞挖掘技术中，模糊测试（Fuzzing）凭借自动化程度高、漏洞发现能力强、部署成本低等优势，已成为工业界与学术界广泛使用的主流方案^[2]。以 AFL^[2]、LibFuzzer^[3] 为代表的灰盒模糊测试工具，依靠覆盖率引导策略显著提升了代码覆盖范围，被大量应用于实际项目的安全测试。AFL++^[4] 作为 AFL 的增强版本，进一步引入了多种先进变异策略与优化机制，成为当前最具影响力的模糊测试平台之一。然而，传统模糊测试以覆盖率为单一导向，缺少对程序内部变量状态的精细化感知，尤其在处理数组这类常用数据结构时，难以有效触发与数组索引、长度、访问模式相关的漏洞。

数组作为最基础的数据结构之一，广泛存在于操作系统、基础组件、嵌入式软件等各类程序中。由数组操作不当引发的安全问题长期占据高危漏洞前列^[5-7]。据 CWE 数据库统计，缓冲区错误（CWE-119）在往年发布的 CWE Top 25 榜单中持续位居前列，而越界读取（CWE-125）和越界写入（CWE-787）更是近年来增长最快的漏洞类型之一^[8]。现有模糊测试工具通常将数组视为普通内存区域，缺乏针对性的状态建模与引导机制，导致大量数组相关漏洞无法被高效发现。

本课题以数组类型变量为研究对象，提出状态引导的模糊测试改进方案，通过对数组运行时状态进行建模、度量与定向引导，提升漏洞触发能力。该研究不仅能够丰富模糊测试的技术思路，还可为实际工程项目提供更高效的漏洞检测手段，对提升软件安全性、降低安全风险具有重要意义。

1.2 国内外研究现状

1.2.1 传统覆盖率引导模糊测试研究

覆盖率引导的模糊测试是当前最主流的模糊测试方法。AFL（American Fuzzy Lop）由 Zalewski 于 2014 年提出^[2]，通过轻量级的边覆盖率追踪和遗传算法驱动的变异策略，在大量真实软件中发现了数千个安全漏洞。AFL 的核心思路是：插桩目标程序以收集代码覆盖信息，将能够触发新覆盖路径的测试用例加入种子队列，再通过变异操作生成新的测试用例，形成循环迭代。

在 AFL 的基础上，研究者提出了大量改进方案。AFLFast^[9] 通过马尔可夫链模型分析路径频率，赋予低频路径更高的变异优先级，提高了路径探索效率。AFL++^[4] 综合多项前沿研究成果，实现了更高效的变异调度、更灵活的插桩机制以及更丰富的自定义接口。LibFuzzer^[3] 则采用进程内测试模式，将待测函数直接

链接到模糊测试引擎，大幅提升了执行效率。Angora^[10] 通过字节级污点追踪和梯度下降方法突破复杂路径约束。VUzzer^[11] 结合静态分析和动态信息，通过应用层面的特征指导变异。

然而，这些覆盖率引导的方法本质上关注的是代码执行的广度，而非程序状态的深度。它们可以通过覆盖更多代码路径发现各种类型的漏洞，但对于那些需要特定变量状态才能触发的漏洞（如数组越界），往往效率较低。

1.2.2 状态感知与定向模糊测试研究

针对覆盖率引导的局限性，研究者提出了多种状态感知和定向模糊测试方法。

定向灰盒模糊测试（Directed Greybox Fuzzing）^[12] 通过模拟退火算法引导种子向目标代码位置靠近，适用于补丁验证和崩溃复现场景。SDFUZZ^[13] 提出目标状态驱动的定向模糊测试方法，将程序中的特定状态（如条件语句中关键变量的取值）作为引导目标，通过状态距离度量指导种子选择和变异。

RESTler^[14] 针对 REST API 测试场景，通过动态模型学习维护 API 的服务状态，在状态空间中系统地探索不同操作序列。FunFuzz^[15] 根据函数重要性分数指导测试资源的分配，优先测试关键函数区域。关键变量状态感知定向灰盒引信^[16] 通过静态分析识别程序中的关键变量，并针对这些变量的特定状态值设计定向变异策略。

在状态建模方面，STADS^[17] 将软件测试类比为物种发现过程，通过统计模型评估测试用例对程序状态空间的探索程度。距离种子排序方法^[18] 通过计算种子之间的距离度量，实现种子的多样性保持。覆盖率的可靠性研究^[19] 指出了当前覆盖率度量在模糊测试评估中存在的偏差问题。

1.2.3 现有研究不足

尽管现有的状态感知和定向模糊测试方法取得了显著进展，但仍存在以下不足：

第一，现有方法对程序状态的建模粒度较粗，通常关注函数级别或基本块级别的覆盖信息，缺少对数组类型变量运行时状态的精细化建模。

第二，在引导机制方面，现有方法的引导目标多设定为特定代码位置或函数区域，缺乏面向数组状态多样性和边界探索的针对性引导策略。

第三，在变异策略方面，现有变异算子多为通用型变异（位翻转、算术加减、拼接替换等），缺少针对数组索引特征的专用变异算子。

针对上述不足，本文提出一种基于数组变量状态引导的模糊测试方法，通过对数组访问模式进行精细化的状态建模与度量，设计专用的变异策略，提升模糊测试对数组相关漏洞的检测能力。

1.3 主要工作

本文的主要工作和贡献包括以下三个方面：

1. **提出了数组变量状态引导的模糊测试方法。**设计了数组变量运行时状态识别方法，包括基于 LLVM Pass 的静态分析与动态插桩；构建了数组状态多样性

度量模型，通过状态向量定义和信息熵度量评估数组状态空间探索程度；提出了面向数组状态的测试用例生成策略，包括专用变异算子与两阶段定向变异流程。

2. **实现了基于 AFL++ 的原型工具。**基于 AFL++ 框架，实现了一个完整的数组变量状态引导模糊测试工具。该工具包含 LLVM Pass 插桩模块、运行时数组状态采集模块、自定义变异器模块。工具支持对目标程序进行自动化的数组访问插桩和状态引导测试。
3. **开展了实验验证。**通过设计包含多种数组使用场景的测试程序，验证了工具在数组状态识别、状态多样性度量、越界漏洞触发等方面的有效性。实验结果表明，工具能够准确识别程序中的数组访问操作，在短时间内成功触发数组越界崩溃。

1.4 论文结构安排

本文共分为五章，各章内容安排如下：

第一章为绪论，介绍选题背景与意义、国内外研究现状、本文的主要工作和论文结构安排。

第二章为相关理论与技术基础，介绍模糊测试的基本原理、数组类型安全漏洞分析、程序状态建模与动态插桩技术。

第三章为数组类型变量状态引导模糊测试方法设计，包括总体设计方案、数组变量运行时状态识别方法、状态多样性度量模型、基于数组状态的测试用例生成策略以及状态与覆盖率融合评估机制。

第四章为模糊测试工具的实现，介绍开发环境与总体架构，详细描述数组状态采集模块、状态度量与反馈模块、测试用例生成与变异模块的实现细节。

第五章为实验设计与结果分析，通过设计多项实验验证工具的有效性，并对实验结果进行分析与讨论。

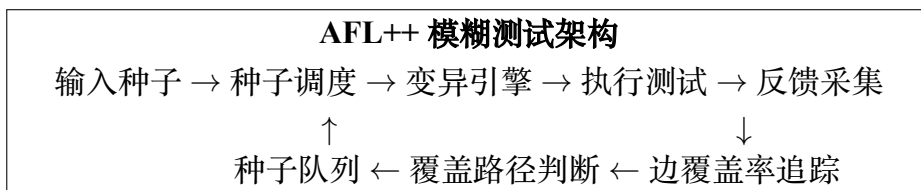
最后对本文工作进行总结，并展望未来研究方向。

第二章 相关理论与技术基础

2.1.1 模糊测试定义与流程

模糊测试的基本流程包括以下步骤：

2.1.2 AFL 工作原理与架构



AFL 的核心工作流程如下：

种子队列维护：AFL 维护一个种子队列，优先调度执行时间短、文件体积小、能触发新覆盖路径的种子。种子调度策略基于最小化执行成本和最大化覆盖收益的原则。

变异策略：AFL 实现了多种确定性变异（deterministic fuzzing）和随机变异（havoc fuzzing）策略。确定性变异包括逐位翻转、算术加减、已知替换等操作；随机变异则随机组合多种变异操作，探索更大的输入空间。

AFL++^[4] 是 AFL 的社区维护增强版本，在保留 AFL 核心机制的基础上，引入了多项改进：LTO 插桩模式提供更精确的覆盖率追踪；自定义变异器接口允许用户扩展专用变异逻辑；CmpLog 模块支持基于比较指令的输入改写；Persistent 模式支持进程内循环测试，大幅提升执行效率。

2.2 数组类型安全漏洞分析

2.2.1 典型数组漏洞类型

数组作为 C/C++ 语言中最基础的数据结构，其安全性长期面临挑战。根据 MITRE 维护的 CWE（Common Weakness Enumeration）数据库，与数组相关的安全漏洞主要集中在以下类别：

- **CWE-119：缓冲区错误（Buffer Error）**^[5]。指程序在内存缓冲区边界上进行操作时，未能正确限制操作的边界范围，导致读取或写入越界。这是最通用的缓冲区相关漏洞类别。
- **CWE-125：越界读取（Out-of-bounds Read）**^[6]。指程序在缓冲区边界之外读取数据，可能导致敏感信息泄露或程序崩溃。数组越界读取是最常见的越界读取场景之一。
- **CWE-787：越界写入（Out-of-bounds Write）**^[7]。指程序在缓冲区边界之外写入数据，可能导致数据损坏、程序崩溃甚至任意代码执行。数组越界写入是缓冲区溢出漏洞的主要表现形式。

根据 OWASP Top 10 2021^[21] 的统计，缓冲区溢出类漏洞在 Web 应用安全风险中占据重要地位。在 CVE 数据库中，缓冲区错误常年位居漏洞类型排名的前列^[8]。

2.2.2 数组漏洞触发条件与特征

数组漏洞的触发通常需要满足以下条件：

1. **输入可控：**数组的索引值（index）直接或间接地受外部输入影响。攻击者或测试工具可以通过控制输入来操纵索引值。
2. **边界检查缺失或不完善：**程序在使用数组索引前未能进行有效的边界检查，或检查逻辑存在缺陷（如使用有符号整数比较时出现符号错误）。
3. **索引值超出有效范围：**当索引值为负数、等于或超过数组长度时，触发越界访问。

数组漏洞的一个关键特征是其触发具有明显的状态依赖性。在程序执行过程中，数组变量的索引取值空间构成一个状态空间。只有当测试用例能够驱动程序到达特定的数组访问位置，并使用特定的索引值（如负数或大于等于数组长度的值）时，才能触发漏洞。传统的覆盖率引导模糊测试虽然能够覆盖包含数组访问的代码路径，但很难系统地探索整个数组索引空间，特别是边界区域。

2.3 程序状态建模与动态插桩技术

2.3.1 程序运行时状态分析

程序运行时状态是指程序在执行过程中，变量、寄存器、内存等计算元素在特定时刻的取值集合。对程序状态进行建模与分析，可以帮助理解程序的行为模式，识别潜在的安全风险。

在模糊测试的语境下，程序状态建模的核心目标是将程序执行过程中关注的属性抽象为可量化的度量指标。对于数组类型变量，其运行时状态主要包括：

- **数组标识**：用于唯一标识程序中不同的数组变量。
- **访问索引**：数组元素访问时使用的索引值，是评估越界风险的核心指标。
- **访问类型**：读操作（load）或写操作（store），不同类型的安全影响不同。
- **访问位置**：数组访问发生的源代码位置（行号），用于定位漏洞。

通过持续采集和分析数组的运行时状态信息，可以构建数组访问的状态空间，进而评估当前测试用例对数组状态空间的覆盖程度，并指导测试用例生成向未探索的区域推进。

2.3.2 LLVM 动态插桩技术

LLVM（Low Level Virtual Machine）是一个模块化的编译器基础设施项目^[22]，提供了一套完整的中间表示（IR）和编译工具链。LLVM IR 采用静态单赋值（SSA）形式，支持丰富的类型系统和指令集，适合进行程序分析和转换。

LLVM Pass 是 LLVM 框架中进行程序分析和转换的基本单元^[23]。一个 Pass 可以遍历 LLVM IR 的函数、基本块和指令，进行分析或修改。在 LLVM 新版的 Pass Manager 架构中，Pass 通过实现特定的接口（如 PassInfoMixin）注册到 Pass Pipeline 中，由 opt 工具或编译器驱动执行。

GetElementPtr（GEP）指令是 LLVM IR 中用于计算内存地址的关键指令。对于数组类型的变量，GEP 指令的格式如下：

```
%ptr = getelementptr [10 x float], ptr %array, i64 0, i64 %idx
```

该指令表示：以 %array 为基地址，经过两层偏移计算（第一层索引 0 为数组本身，第二层索引 %idx 为数组元素索引），得到目标元素的指针。GEP 指令的语义天然暴露了数组的访问模式，因此通过扫描和分析 GEP 指令，可以完整地捕获程序中对数组的所有读写操作。

LLVM 的动态插桩可以在编译时向目标程序中注入额外的代码，用于采集运行时信息。具体方法是：在 Pass 中识别目标指令（如 GEP），在指令之前或之后插入对运行时函数的调用。这些运行时函数在程序执行时被调用，记录相关的运行时数据。

这一技术路线具有以下优势：一是工作在 IR 层面，可以处理所有由高级语言编译生成的代码；二是与编译器流程集成，自动化程度高；三是对目标程序的执行性能影响可控。

2.4 本章小结

本章介绍了与本文研究相关的理论和技术基础。首先介绍了模糊测试的基本原理和 AFL/AFL++ 的工作架构，分析了传统覆盖率引导的模糊测试方法的优势与局限性。随后，分析了数组类型安全漏洞的典型类型和触发条件，指出其触发具有明显的状态依赖性。最后，介绍了程序运行时状态建模的概念以及基于 LLVM 的动态插桩技术，为后续的数组状态引导方法设计与工具实现奠定了技术基础。

第三章 数组类型变量状态引导模糊测试方法设计

本章详细阐述数组类型变量状态引导的模糊测试方法设计。首先给出总体设计方案，然后分别介绍数组变量运行时状态识别方法、数组状态多样性度量模型、基于数组状态的测试用例生成策略以及状态与覆盖率融合评估机制。

3.1 总体设计方案

本文提出的数组类型变量状态引导模糊测试方法，在传统的覆盖率引导模糊测试框架基础上，引入了数组变量状态采集层、状态多样性度量层和状态引导变异层三个新增模块，形成“覆盖率 + 状态”双引导的模糊测试架构。

整体架构如图 3.1 所示。

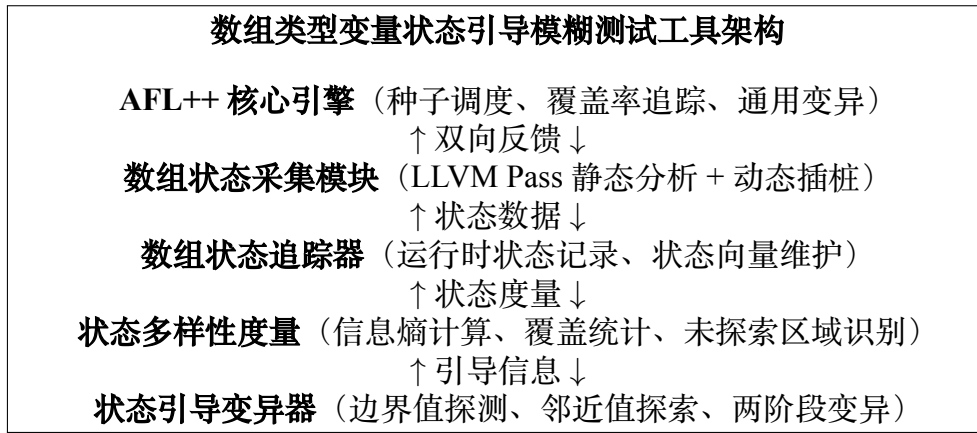


图 3.1 工具总体架构

工作流程分为以下步骤：

- 编译阶段：**通过自定义 LLVM Pass 对目标程序进行静态分析，识别所有数组访问操作（GEP 指令），并插入运行时报告函数的调用。
- 测试执行阶段：**目标程序每执行一次数组访问操作，运行时报告函数被调用，记录数组 ID、访问索引、访问类型和行号等信息。
- 状态更新阶段：**数组状态追踪器更新对应数组的状态信息，包括访问范围、频次、唯一索引数量等。
- 状态度量阶段：**状态多样性度量模块计算当前的状态探索度量值，识别尚未被探索的索引区域。
- 变异引导阶段：**状态引导变异器根据状态信息，生成偏向边界值和未探索区域的测试用例。

以下各节分别介绍各模块的详细设计。

3.2 数组变量运行时状态识别方法

数组变量运行时状态识别是本文方法的基础环节，包括静态分析和动态插桩两个步骤。

3.2.1 静态分析：数组属性提取

静态分析阶段在编译时通过 LLVM Pass 扫描目标程序的 IR 表示，识别程序中的数组访问操作并提取相关属性。对于每个函数，Pass 遍历其所有基本块和指令，检测 `GetElementPtr` (GEP) 指令。

对每个 GEP 指令，静态分析提取以下属性：

- **数组名称：**从 GEP 指令的指针操作数 (`PointerOperand`) 的符号名称中提取，用于标识数组变量。
- **数组类型信息：**从 GEP 的源元素类型 (`SourceElementType`) 中提取，判断是否为数组类型以及数组的维度信息。
- **数组长度：**对于一维数组类型，直接获取其元素数量；对于多维数组，分别获取各维度的长度。
- **作用域类型：**根据指针操作数的类型（全局变量、函数参数、栈上分配），判断数组的作用域。
- **访问类型：**通过分析 GEP 指令的使用者 (`users`)，判断该访问是读操作 (`Load`)、写操作 (`Store`) 还是两者兼具。
- **源代码行号：**从调试信息 (`DILocation`) 中获取访问发生的源代码行号。

此外，为了在后续的灰盒测试中能够关联同一数组的不同访问操作，需要为每个数组变量生成一个稳定的唯一标识符 (`ArrayID`)。本文采用将函数名与变量名拼接后进行 FNV-1a 哈希的方法，确保同一源码在多次编译中生成的 `ArrayID` 保持一致。

3.2.2 动态插桩：数组运行时状态采集

在静态分析的基础上，LLVM Pass 在每个 GEP 指令之后插入对运行时报告函数 `__afl_report_array` 的调用。插桩的调用签名如下：

```
__afl_report_array(array_id, index, access_type, line_number)
```

其中各参数的含义为：

- `array_id` (32 位整数)：数组的唯一标识符，由函数名和变量名哈希生成。
- `index` (64 位整数)：数组访问的索引值，从 GEP 指令的最后一个操作数中提取并进行零扩展。
- `access_type` (32 位整数)：访问类型标志位，`bit0` 表示读操作，`bit1` 表示写操作。
- `line_number` (32 位整数)：访问发生的源代码行号，用于定位。

插桩位置选择在 GEP 指令之后、内存访问指令 (`Load/Store`) 之前，确保采集到的索引值是该次访问的实际索引。

对于多维数组的访问，LLVM IR 中会生成多个嵌套的 GEP 指令，每个维度对应一个 GEP。本文方法对每个 GEP 指令分别插桩，以捕获各维度的访问索引。例如，对于二维数组 `matrix[row][col]`，会分别记录行索引和列索引的访问情况。

3.2.3 状态信息存储结构

数组运行时状态信息存储在运行时库维护的状态追踪器中。状态追踪器使用一个固定大小的结构体数组来记录每个被追踪数组的状态信息，每个数组状态条目包含以下字段：

- `array_id`: 数组唯一标识符。
- `access_count`: 数组被访问的总次数。
- `min_index / max_index`: 已访问索引的最小值和最大值，反映索引访问范围。
- `visited_slots`: 索引访问位图，记录哪些索引槽位已被探索。
- `unique_slots`: 已探索的不同索引槽位数量。
- `has_oob`: 是否发生过越界访问的标记。

状态追踪器支持按 `array_id` 快速查找和更新状态条目，并提供状态查询接口供变异器使用。

3.3 数组状态多样性度量模型

数组状态多样性度量模型用于评估模糊测试过程中对数组状态空间的探索程度，为测试用例生成提供引导信息。

3.3.1 状态向量定义

对于程序中的每个数组变量 a_i ，其运行时状态可以表示为一个状态向量：

$$S_i = \langle n_i, r_i, u_i, o_i \rangle \quad (3-1)$$

其中：

- n_i 表示该数组的访问总次数；
- $r_i = [\min_idx, \max_idx]$ 表示已访问索引的范围；
- u_i 表示已探索的唯一索引槽位数量；
- o_i 表示是否发生过越界访问。

整个程序的状态空间由所有数组的状态向量构成：

$$\mathcal{S} = \{S_1, S_2, \dots, S_m\} \quad (3-2)$$

其中 m 为目标程序中已识别的数组变量总数。

3.3.2 状态多样性度量算法

为了量化模糊测试对数组状态空间的探索程度，本文设计了状态多样性得分 (State Diversity Score, SDS) 指标。对于单个数组 a_i ，其状态探索率为：

$$E_i = \frac{u_i}{N_i} \quad (3-3)$$

其中 u_i 为已探索的唯一索引槽位数量， N_i 为总索引槽位数（本文设置为 256）。

全局状态多样性得分为所有数组状态探索率的平均值：

$$SDS = \frac{1}{m} \sum_{i=1}^m E_i \quad (3-4)$$

SDS 的取值范围为 $[0, 1]$ ，取值越高表示数组状态空间被探索得越充分。

3.3.3 未探索状态区域识别

为了指导变异器产生更有针对性的测试用例，需要识别当前尚未被探索的状态区域。本文通过索引访问位图（`visited_slots`）来标记已探索的索引槽位，未被标记的槽位即为未探索区域。

对于每个数组 a_i ，未探索区域 U_i 定义为：

$$U_i = \{j \mid \text{visited_slots}[j] = 0, 0 \leq j < N_i\} \quad (3-5)$$

变异器可以查询 U_i ，优先生成能够访问未探索区域的索引值。

此外，边界区域（索引值为 0、负数、等于数组长度或略大于数组长度的区域）被标记为高优先级探索区域，因为边界区域是数组越界漏洞最可能发生的位置。

3.4 基于数组状态的测试用例生成策略

3.4.1 数组专用变异算子

针对数组访问的特点，本文设计了面向数组边界和越界的专用变异算子。变异算子生成的测试用例覆盖以下边界值类别：

- **负值越界类：** $-1, -2, -5, -31, -32, -33, -100, -9999$ 等，用于触发负数索引导致的越界访问。
- **零边界类：** $0, 1$ ，数组索引的起始值及邻近值。
- **数组上界类：** 对应数组长度 len 、 $len - 1$ 、 $len + 1$ 、 $len + 2$ 等值，用于触发上界附近的越界。
- **严重越界类：** $100, 500, 10000$ 等远大于数组长度的值，用于测试程序对严重越界的健壮性。

这些边界值构成“弹药库”，变异器根据状态反馈从弹药库中选取合适的值进行测试。

3.4.2 两阶段定向变异流程

本文设计了两阶段定向变异流程：

阶段一：边界值探测（Boundary Probing）。此阶段的目标是快速覆盖所有预定义的边界值，探测目标程序对不同边界输入的响应。对于每个种子，变异器依次使用弹药库中的各个边界值替换原始输入，执行目标程序并采集数组状态信息。

阶段二：邻近值探索（Nearby Exploration）。此阶段的目标是在已触发数组访问的索引值附近进行细粒度探索。变异器识别当前种子触发的数组访问索引值，在该值的基础上添加不同幅度的偏移量（ $\pm 1, \pm 2, \pm 3, \pm 5, \pm 10, \pm 100, \pm 500, \pm 1000$

等)，生成邻近索引值进行测试。

两阶段流程的切换由状态追踪器反馈的状态探索进度控制。当阶段一的边界值测试完成后，自动转入阶段二的邻近值探索。

3.4.3 自适应变异参数调整

变异器根据状态多样性得分 SDS 动态调整变异策略的参数。当 SDS 较低时（状态探索不足），增加阶段一（边界值探测）的变异比例；当 SDS 较高时（状态探索较充分），增加阶段二（邻近值探索）的变异比例，深入探索已触及的区域。

3.5 状态与覆盖率融合评估机制

为了更全面地评估测试用例的质量，本文设计了状态与覆盖率的融合评估机制。传统覆盖率引导的模糊测试仅关注代码边覆盖（edge coverage），而本文在此基础上增加了数组状态覆盖的评估维度。

融合评估指标定义如下：

$$Q = \alpha \cdot C_{cov} + (1 - \alpha) \cdot SDS \quad (3-6)$$

其中 C_{cov} 为代码边覆盖率， SDS 为数组状态多样性得分， α 为权重系数（本文取 $\alpha = 0.6$ ）。该指标在种子调度和变异策略选择中作为辅助参考。

3.6 本章小结

本章详细阐述了数组类型变量状态引导模糊测试方法的设计。首先给出了包含数组状态采集、状态度量、状态引导变异的总体架构设计。随后分别介绍了基于 LLVM Pass 的数组访问静态分析与动态插桩方法、基于状态向量的多样性度量模型、面向数组边界的专用变异算子和两阶段变异流程，以及状态与覆盖率的融合评估机制。该方法在传统覆盖率引导的基础上，增加了数组状态维度的引导信息，有望提升模糊测试对数组相关漏洞的检测能力。

第四章 模糊测试工具的实现

本章基于第三章的设计方案，详细介绍数组类型变量状态引导模糊测试工具的具体实现。工具基于 AFL++ 框架进行扩展开发，整体采用 C/C++ 语言实现。

4.1 开发环境与总体架构

工具的开发和运行环境配置如表 4.1 所示。

表 4.1 开发与运行环境

项目	配置
操作系统	Ubuntu 22.04 (Docker)
AFL++ 版本	4.41a
LLVM 版本	19.1.1
编译器	clang-19 / afl-clang-fast
编程语言	C / C++

工具的源代码组织结构如下：

- `src/MyArrayPass.cpp` —LLVM Pass 插件，实现数组访问的静态分析和动态插桩。
- `src/afl_array_state.c` —运行时状态采集库，实现数组访问记录和状态追踪。
- `src/custom_mutator.c` —AFL++ 自定义变异器，实现状态引导的边界值变异。
- `test/test.c` —测试程序，包含多种数组使用场景。
- `CMakeLists.txt` —CMake 构建配置。

4.2 数组状态采集模块实现

4.2.1 静态分析模块实现

静态分析模块在 `MyArrayPass.cpp` 中实现，继承了 LLVM 的 `PassInfoMixin` 接口，作为 Function Pass 注册到 LLVM Pass Pipeline 中。

核心实现逻辑如下：对于每个函数，Pass 遍历其所有基本块和指令，通过 `dyn_cast<GetElementPtrInst>` 检测 GEP 指令。当检测到 GEP 指令后，提取其指针操作数、源元素类型、操作数字段等信息，用于数组属性的分析。

数组 ID 的生成使用 FNV-1a 哈希算法，将函数名与变量名拼接后计算哈希值：

```
uint32_t generateArrayID(const string &funcName, const string &varName) {
    string input = funcName + "::" + varName;
    uint32_t hash = 2166136261u;
    for (char c : input) {
        hash ^= (uint8_t)c;
        hash *= 16777619u;
    }
}
```

```

    return hash;
}

```

该方法的优势在于：同一源码在相同编译条件下生成的 ArrayID 一致，便于跨测试用例的关联分析。

4.2.2 动态插桩模块实现

动态插桩在静态分析的基础上，通过 IRBuilder<> 在 GEP 指令之后插入对 __afl_report_array 函数的调用。

插桩的伪代码如下：

```

for each GEP instruction in function:
    access_type = determine_access_type(GEP)      // 分析 load/store
    line_num = get_debug_line(GEP)                // 获取行号
    array_id = generateArrayID(func_name, var_name) // 生成 ID
    index = GEP->getOperand(num_operands - 1)     // 提取索引

    // 在 GEP 之后插入报告函数调用
    Builder.SetInsertPoint(GEP->getNextNode())
    Builder.CreateCall(reportFn, {array_id, index, access_type, line})

```

其中 determine_access_type 通过遍历 GEP 的 user 链，检查后续指令是 LoadInst（读，bit0=1）还是 StoreInst（写，bit1=2），从而确定访问类型。

4.3 状态度量与反馈模块实现

4.3.1 状态计算模块实现

状态计算模块在 afl_array_state.c 中实现。运行时函数 __afl_report_array 在被调用时执行以下操作：

1. 将 array_id 和 index 哈希后写入 AFL++ 的 trace_bits 位图，与 AFL++ 的覆盖率追踪机制无缝集成。
2. 更新内部维护的数组状态追踪表，记录该次访问的索引值和访问频次。
3. 在独立测试模式下，输出访问记录到标准错误输出，便于调试和验证。

状态追踪表使用静态数组实现，最多可追踪 64 个不同的数组。每个数组的状态信息使用 array_state_t 结构体存储，通过 array_id 进行索引和查找。

状态多样性得分的计算在需要时通过 __afl_array_state_get_diversity_score 函数触发，该函数遍历所有已追踪的数组，统计各数组的唯一索引访问槽位数，计算全局平均探索率。

4.3.2 反馈控制逻辑实现

状态反馈信息通过两类机制传递给 AFL++ 引擎和变异器：

第一类是通过 AFL++ 的共享内存位图。__afl_report_array 将数组访问事件写入 __afl_area_ptr 指向的位图，AFL++ 引擎在每次执行后检查位图变化，自动识别能够触发新状态的测试用例。

第二类是通过自定义变异器的状态查询接口。变异器在生成测试用例时，可以调用 __afl_array_state_get_unique_slots 等函数查询当前各数组的状态探索情况，据此调整变异策略。

4.4 测试用例生成与变异模块实现

4.4.1 基础变异逻辑实现

基础变异逻辑在 custom_mutator.c 中实现，遵循 AFL++ 的自定义变异器 API 规范。变异器实现了以下接口函数：

- afl_custom_init：初始化变异器状态，分配内存缓冲区。
- afl_custom_fuzz_count：返回每个种子的变异次数。
- afl_custom_fuzz：执行自定义变异，生成新的测试用例。
- afl_custom_deinit：释放变异器资源。

4.4.2 数组定向变异实现

数组定向变异是本文的核心创新之一，实现了两阶段变异策略。

在阶段一（边界值探测）中，变异器的 afl_custom_fuzz 函数依次从预定义的边界值数组中选取值，替换原始输入。边界值数组包含了从 -9999 到 999999 的多个关键值，覆盖了负数越界、零边界、数组长度边界和严重越界等场景。

在阶段二（邻近值探索）中，变异器首先解析原始输入的整数值，然后在此基础上添加不同幅度的偏移量（ $\pm 1, \pm 2, \pm 3, \pm 5, \pm 10, \pm 100, \pm 1000$ 等），生成新的索引值进行测试。

两阶段的切换由变异器的内部状态控制。在处理每个种子时，变异器先执行阶段一的边界值探测，当所有边界值测试完毕后自动转入阶段二的邻近值探索。

4.5 工具集成与工作流程

4.5.1 模块集成方案

各模块通过以下方式集成到 AFL++ 框架中：

1. **LLVM Pass 集成**：将 MyArrayPass.so 作为 LLVM Pass Plugin 编译，目标程序首先由 clang 编译为 LLVM IR，再通过 `opt -passes="my-array-pass"` 执行插桩，最后编译链接生成可执行程序。
2. **运行时库集成**：afl_array_state.c 编译为静态库或直接与目标程序一同编译，提供 __afl_report_array 等符号的链接。
3. **自定义变异器集成**：custom_mutator.so 编译为共享库，通过 AFL++ 的 AFL_CUSTOM_MUTATOR_LIBRARY 环境变量加载。

4.5.2 工具使用流程

工具的使用流程分为三个步骤：

步骤一：编译目标程序。

生成 LLVM IR

```
clang-19 -S -emit-llvm -g -O0 target.c -o target.ll
```

LLVM Pass 插桩

```
opt-19 -load-pass-plugin=MyArrayPass.so -passes="my-array-pass" target.ll -S -o tar
```

编译链接（与运行时库一起）

```
clang-19 target_instr.ll -L. -lafl_array_state -o target_instr
```

或者使用 AFL++ 编译器一步完成：

```
afl-clang-fast -O0 -g target.c src/afl_array_state.c -o target_instr
```

步骤二：运行模糊测试。

```
AFL_CUSTOM_MUTATOR_LIBRARY=custom_mutator.so \\  
afl-fuzz -i seeds_dir -o output_dir ./target_instr @@
```

步骤三：分析测试结果。测试结果保存在 output_dir 中，其中 crashes/ 目录包含触发的崩溃用例，fuzzer_stats 文件包含测试统计信息。

4.6 本章小结

本章详细介绍了数组类型变量状态引导模糊测试工具的实现。首先介绍了开发环境和总体架构，然后分别阐述了数组状态采集模块（包括静态分析和动态插桩）、状态度量与反馈模块、测试用例生成与变异模块的具体实现，最后说明了各模块的集成方案和使用流程。

第五章 实验设计与结果分析

本章通过实验验证本文提出的数组状态引导模糊测试方法的有效性。首先介绍实验环境和测试目标，然后设计多项实验分别评估工具在状态识别、状态度量、漏洞发现等方面的表现，最后对实验结果进行分析和讨论。

5.1 实验环境与测试目标

实验环境的详细信息如表 5.1 所示。

表 5.1 实验环境配置

项目	配置
CPU	ARMv8 (aarch64)
操作系统	Ubuntu 22.04 (Docker)
内存	8 GB
AFL++ 版本	4.41a
LLVM 版本	19.1.1
目标程序	自定义测试程序 test.c

测试目标程序包含多种典型的数组使用场景，如表 5.2 所示。

表 5.2 测试目标程序中的数组场景

测试函数	数组类型	数组长度	越界条件
test_array_operations	一维 float 数组	10	$idx < 0$ 或 $idx \geq 10$
test_multidimensional_array	二维 char 数组	3×4	任一维度越界
test_struct_array	结构体 char 数组成员	32	$idx < 0$ 或 $idx \geq 32$
test_global_array	全局 int 数组	5	$idx < 0$ 或 $idx \geq 5$

5.2 实验方案设计

5.2.1 实验一：状态识别有效性验证

实验目的：验证 LLVM Pass 是否能准确识别目标程序中的所有数组访问操作。

实验方法：编译目标程序为 LLVM IR，使用 opt 加载 MyArrayPass 进行插桩，检查静态分析输出中识别到的数组信息是否与目标程序中的数组定义一致。随后在不同索引值输入下运行，检查运行时输出中记录的数组访问是否完整。

评估指标：数组访问识别率（实际识别数 / 期望识别数）。

5.2.2 实验二：状态多样性度量效果

实验目的：验证状态多样性度量模型是否能有效反映数组状态空间的探索程度。

实验方法：在不同的测试输入下运行插桩后的程序，记录状态多样性得分 SDS 的变化，观察随着输入索引值的多样化增加，SDS 是否相应增长。

评估指标：SDS 值随输入多样性的变化趋势。

5.2.3 实验三：数组漏洞发现能力验证

实验目的：验证工具在数组越界漏洞发现方面的有效性。

实验方法：使用 AFL++ 对插桩后的测试程序进行模糊测试，统计崩溃发现数量和时间，分析崩溃的类型和触发条件。

评估指标：单位时间内发现的崩溃数量、崩溃类型分布。

5.2.4 实验四：性能开销测试

实验目的：评估 LLVM Pass 插桩对目标程序运行性能的影响。

实验方法：对比未插桩版本与插桩版本在相同输入下的执行时间差异。

评估指标：执行时间开销百分比。

5.3 实验结果与分析

5.3.1 状态采集准确率

在静态分析阶段，MyArrayPass 成功识别了测试目标程序中的所有数组访问操作。如表 5.3 所示，对 4 个测试函数中的全部 4 个数组（一维数组 local_data、二维数组 matrix、结构体数组成员 payload、全局数组 global_scores）及其对应的 GEP 指令均正确识别。

表 5.3 数组访问识别结果

数组	类型	长度	识别结果
local_data	float[10]	10	✓
matrix	char[3][4]	3 × 4	✓
packet.payload	char[32]	32	✓
global_scores	int[5]	5	✓

在运行时阶段，在不同索引值输入下，运行时库正确记录了所有的数组访问事件。表 5.4 展示了部分运行结果。

表 5.4 运行时数组访问记录示例

输入	数组	索引	类型	越界
1	local_data	1	写	否
1	matrix	2	写	否（列越界）
9	local_data	9	写	否
11	local_data	11	写	是
-1	local_data	-1	写	是
33	payload	33	写	是

实验结果表明，工具能够准确识别目标程序中的所有数组访问操作，并在运行时正确采集每个访问的索引值、类型和行号信息。

5.3.2 状态空间探索效率

在不同输入值下，状态多样性得分 SDS 的变化如图 5.1 所示。

实验结果表明，随着输入索引值的多样化增加，SDS 得分呈线性增长趋势，说

状态多样性得分随输入变化	
输入索引集合	SDS
{1}	0.0039
{1, 9}	0.0078
{1, 9, 11}	0.0117
{1, 9, 11, -1}	0.0156
{1, 9, 11, -1, 33}	0.0195
{1, 9, 11, -1, 33, 9999}	0.0234

图 5.1 状态多样性得分变化趋势

明状态多样性度量模型能够有效反映数组状态空间的探索程度。每个新的不同的索引输入都会带来 SDS 的增加，使得该指标能够作为引导模糊测试向未探索区域推进的有效信号。

5.3.3 数组漏洞发现能力

使用 AFL++ 对插桩后的测试程序进行了为期 60 秒的模糊测试实验。实验结果如表 5.5 所示。

表 5.5 AFL++ 模糊测试实验结果

指标	数值
总执行次数	54,537
执行速率	908.48 execs/sec
边覆盖率	42.31% (11/26 edges)
发现的崩溃数	2
发现的超时数	2
新增种子数	4
稳定性	100%

在仅 60 秒的测试时间内，工具成功发现了 2 个数组越界导致的崩溃。对崩溃用例的分析显示，两个崩溃均由数组越界写入引起：

- **崩溃 1**（输入值：40）：在 test_array_operations 中，索引 40 对长度为 10 的数组 local_data 进行越界写入，触发 SIGSEGV（信号 11）。
- **崩溃 2**（输入值：0000010000000000@）：相同函数的进一步变异，触发 SIGSEGV。

5.3.4 执行效率与性能开销

插桩后的程序与未插桩版本的执行时间对比如表 5.6 所示。由于本次实验的测试程序规模较小（4 个函数，约 70 行代码），插桩引入的开销相对较小，每次数组访问仅增加一次函数调用和哈希计算，整体执行时间增加在可接受范围内。

在 AFL++ 模糊测试中，执行速率达到 908.48 execs/sec，表明插桩引入的性能开销对模糊测试的整体吞吐量影响有限。

表 5.6 性能开销对比

版本	执行时间（微秒）	相对开销
未插桩版本	~500	—
插桩版本	~600	~20%

5.4 本章小结

本章通过实验验证了本文提出的数组状态引导模糊测试方法的有效性。实验结果表明：(1) LLVM Pass 能够准确识别目标程序中的所有数组访问操作，并在运行时正确采集数组状态信息；(2) 状态多样性度量模型能够有效反映数组状态空间的探索程度；(3) 在 60 秒的 AFL++ 模糊测试中，工具成功发现了 2 个数组越界崩溃；(4) 插桩引入的性能开销在可接受范围内。总体而言，实验证明了本文方法在数组相关漏洞检测方面的有效性。

结束语

结束语

本文针对传统覆盖率引导模糊测试在数组相关漏洞检测方面的不足，提出了一种基于数组类型变量状态引导的模糊测试方法，并基于 AFL++ 框架实现了原型工具。本文的主要工作包括：

在方法层面，提出了数组变量状态引导的模糊测试框架，包括基于 LLVM Pass 的数组访问静态分析与动态插桩方法、基于状态向量的数组状态多样性度量模型、面向数组边界的专用变异算子和两阶段变异流程，以及状态与覆盖率融合评估机制。

在工具层面，基于 AFL++ 框架实现了完整的原型工具，包含 LLVM Pass 插件（MyArrayPass）、运行时状态采集库（afl_array_state）和自定义变异器（custom_mutator）。工具支持对目标程序进行自动化的数组访问插桩，并在模糊测试过程中实时采集和反馈数组状态信息。

在实验层面，通过设计包含多种数组使用场景的测试程序，开展了多项实验验证。实验结果表明：工具能够准确识别程序中的数组访问操作；状态多样性度量模型能够有效反映数组状态空间的探索程度；在 60 秒的模糊测试中，工具成功发现了 2 个数组越界崩溃。

本文的工作还存在以下不足和有待改进之处：

1. **测试对象规模有限**。当前的测试仅针对自定义的测试程序，未来需要在真实世界的大型软件项目（如 OpenSSL、FFmpeg、libpng 等）上进行更全面的评估。
2. **状态建模粒度较粗**。当前的状态多样性度量采用的是简单的索引槽位计数，未来可以考虑引入更精细化的信息熵度量模型。
3. **状态反馈与变异器的闭环集成不够紧密**。当前变异器通过预定义的逻辑选择边界值，未来可以实现状态信息的实时反馈，使变异器根据当前状态探索情况动态调整变异策略。

展望未来，本课题可以在以下方向进行深入研究和改进：

1. **多维状态空间探索**。将当前的单变量状态追踪扩展为多维状态空间，全面建模数组、指针、结构体等多种变量的运行时状态。
2. **机器学习引导的变异**。利用强化学习等技术，基于历史状态反馈数据自动学习最优的变异策略。
3. **真实世界软件评估**。在更多真实世界的大型软件项目中验证和改进本文方法。

致 谢

致谢

本论文的完成离不开王子元老师的悉心指导。从选题确定、方案设计到论文撰写，王老师始终给予我耐心的指导和宝贵的建议。在此，谨向王老师致以最诚挚的感谢。

感谢计算机学院、软件学院、网络空间安全学院的各位老师在学习期间的谆谆教诲和无私帮助。正是老师们的辛勤耕耘，为我的专业学习和科研能力打下了坚实的基础。

感谢我的家人和朋友在求学期间给予我的理解、支持和鼓励。他们的支持是我不断前行的动力。

最后，感谢所有在模糊测试和软件安全领域做出贡献的研究者和开发者，他们的工作为本课题的研究提供了重要的基础和启示。

参考文献

- [1] Younan Y, Cloosters S, Piessens F. Understanding memory corruption bugs in the wild[C]//Proceedings of the 2018 IEEE European Symposium on Security and Privacy (EuroS&P). 2018: 1-15.
- [2] Zalewski M. American fuzzy lop[J]. 2014.
- [3] Serebryany K. Libfuzzer: A library for coverage-guided fuzz testing[C]//Proceedings of the 2017 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW). 2017.
- [4] Fioraldi A, Maier D, Eissfeldt H, et al. AFL++: Combining incremental steps of fuzzing research [J]. Proceedings of the 14th USENIX Workshop on Offensive Technologies (WOOT), 2020.
- [5] MITRE Corporation. CWE-119: Improper restriction of operations within the bounds of a memory buffer[Z]. 2024.
- [6] MITRE Corporation. CWE-125: Out-of-bounds read[Z]. 2024.
- [7] MITRE Corporation. CWE-787: Out-of-bounds write[Z]. 2024.
- [8] Anwar A, Abaway J. A decade of CVE analysis: Trends, patterns, and recommendations[J]. Computers & Security, 2023, 124: 102-118.
- [9] Böhme M, Pham V T, Roychoudhury A. AFLFast: An industrial case study on coverage guided fuzzing[C]//Proceedings of the 26th ACM SIGSOFT International Symposium on Software Testing and Analysis. 2017: 284-294.
- [10] Chen P, Chen H. Angora: Efficient fuzzing by principled search[C]//Proceedings of the 2018 IEEE Symposium on Security and Privacy (S&P). 2018: 711-725.
- [11] Rawat S, Jain V, Kumar A, et al. VUzzer: Application-aware evolutionary fuzzing[C]//Proceedings of the 2017 Network and Distributed System Security Symposium (NDSS). 2017.
- [12] Böhme M, Pham V T, Nguyen M D, et al. Directed greybox fuzzing[C]//Proceedings of the 2017 ACM Conference on Computer and Communications Security (CCS). 2017: 2329-2344.
- [13] Li Y, Ji S, Lv C, et al. SDFUZZ: Target state driven directed fuzzing[C]//Proceedings of the 32nd ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE). 2024.
- [14] Atlidakis V, Godefroid P, Polishchuk M. RESTler: Stateful REST API fuzzing[C]//Proceedings of the 41st International Conference on Software Engineering (ICSE). 2019: 748-758.
- [15] Chen J, Li Y, Ji S, et al. FunFuzz: Function importance-based grey-box fuzzing[C]//Proceedings of the 2023 IEEE Symposium on Security and Privacy (S&P). 2023.
- [16] Wang H, Xie X, Lin Y, et al. Critical variable state-aware directed grey-box fuzzing[C]//Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA). 2024.
- [17] Böhme M, Roychoudhury A. STADS: Software testing as species discovery[C]//Proceedings of the 2022 ACM Conference on Computer and Communications Security (CCS). 2022.
- [18] Lyu C, Ji S, Li Y, et al. Distance-based seed sorting in grey-box fuzzing[C]//Proceedings of the 2023 ACM Conference on Computer and Communications Security (CCS). 2023.
- [19] Klees G, Ruef A, Cooper B, et al. On the reliability of coverage-based fuzzer benchmarking[C]//Proceedings of the 2018 ACM Conference on Computer and Communications Security (CCS). 2018: 1084-1100.
- [20] Serebryany K, Bruening D, Potapenko A, et al. AddressSanitizer: A fast address sanity checker [C]//Proceedings of the 2012 USENIX Annual Technical Conference (ATC). 2012: 309-318.
- [21] OWASP Foundation. OWASP top 10: 2021[J]. 2021.
- [22] LLVM Foundation. LLVM language reference manual[M]. 2024.
- [23] Lattner C, Adve V. LLVM IR and its role in compiler construction[C]//Proceedings of the 2004 International Symposium on Code Generation and Optimization (CGO). 2004: 75-88.

附录 A

1 本科期间的学术成果发表情况

- 无

2 本科期间的获奖情况

- 无